AI Based Self Healing IT Infrastructure Framework

Manasa Visakai S
Department of Computer Science
Rajalakshmi Engineering College
Anna University, India
manasasrid30@gmail.com

V Mahalakshmi
Department of Computer Science
Rajalakshmi Engineering College
Anna University, India
vmahashree12@gmail.com

Maria Priscilla M
Department of Computer Science
Rajalakshmi Engineering College
Anna University, India
mariapriscillathiraviyam@gmail.com

Magesh Vasan M
Department of Computer Science
Rajalakshmi Engineering College
Anna University, India
mageshvasan2006@gmail.com

Aswana Lal
Department of Computer Science
Rajalakshmi Engineering College
Anna University, India
aswanalal@rajalakshmi.edu.in

***Abstract*—IT infrastructures today are distributed over many components and have grown quite complex, so watching them continuously and fixing problems by hand is not easy. When services fail, resources get used up, the system behaves strangely or one fault triggers another, availability drops and recovery takes longer. This paper presents an AI based self-healing approach for infrastructure in which monitoring, anomaly detection, failure prediction, root cause analysis and controlled recovery are handled together. The framework keeps collecting telemetry such as CPU load, memory consumption, disk utilization, network performance, response time, container health and system logs, and uses it to find the present state of the infrastructure. We use Isolation Forest to detect anomalies and XGBoost to predict failures. After the possible failure modes are ranked, a Decision Tree is used with metric and log based rules to find the likely reason for the failure, and recovery actions are started according to the cause that was diagnosed. An incident dashboard and a healing history are also kept up to date for the infrastructure, which supports monitoring and decision making. Since failure detection and prediction, root cause analysis and controlled recovery with service verification are combined, the approach is expected to reduce intervention time and improve the reliability of modern IT infrastructure.**

***Index Terms*—AI-based self-healing, IT infrastructure, anomaly detection, failure prediction, root-cause analysis, automated recovery, Isolation Forest, XGBoost, infrastructure monitoring, AIOps.**

I. Introduction

Most IT infrastructure now is distributed. Web services, APIs, databases, worker services, containers and network resources run together so that the overall functionality is available without breaks. As more features and components get added, managing the infrastructure becomes harder. Performance can drop, parts can fail, resources may not be enough, the system can behave in ways nobody expected, and sometimes one problem leads to another until the infrastructure stops working.

In the usual way of managing infrastructure, admins depend on monitoring dashboards, alerts, metrics and logs to find out that an incident has happened. All the needed information is present in these tools, but they do not take the next step of recovering the infrastructure automatically. The administrator has to look at the dashboards and alerts, relate them to each other, find the root cause, decide on a recovery procedure and then apply it, for example by restarting or reconfiguring the services that are faulty. This is difficult because the infrastructure is split across many components and still has to work as one system. Monitoring based on thresholds is also not very effective, as it gives false positives and can miss small incidents. Certain events happen rarely and are not supervised well, so it is hard to define their failure patterns or recover from them. In systems where components depend on each other, finding which component failed, what type of incident it is and which recovery procedure to use can take a lot of effort. So a self-healing system that works well must take care of detection, root-cause analysis and verification of the recovery procedure that was applied.

Self-healing based on AI can make the management and automation of distributed IT infrastructure simpler. Our framework uses the Monitor-Analyze-Plan-Execute (MAPE-K) methodology. Under this methodology the infrastructure is monitored, the telemetry received is analyzed, the failure is identified, the actions required to bring back normal operation are planned, and those actions are executed with as little risk as possible. Isolation Forest detects anomalies with the help of isolation trees, and we use it to find outliers. The XGBoost library is used to predict the failure type.

The framework first relates the detected evidence and the predicted failure type to a group of possible recovery procedures. From these it suggests the best options, which lowers the risk of running a procedure that is not relevant. Then it validates the recovery procedure by checking if the infrastructure has actually healed. Detected incidents are shown on an incident dashboard. A recovery history dashboard can also be part of the framework, and it keeps track of the healing procedures done by the framework as well as by administrators. The target services are web/APIs, databases, workers and containers, though the framework can be used for other infrastructure components as well. Its main focus is continuous monitoring and management of the infrastructure, and it does not carry out any action that is unauthorized or unlimited.

Monitoring dashboards, alerts, logs and metrics only give a baseline view of what happens inside the IT infrastructure, and this knowledge is not used to recover from events automatically.

Our framework extends open source solutions that are already available, so that infrastructure recovery can be automated from the root cause analysis stage onwards with limited help from the administrator.

II. Literature Review

The idea of self healing in computing systems comes from autonomic computing. Kephart and Chess used this term for systems that observe, analyse, plan and act with very little human involvement [3]. They wanted a way to handle the complexity of large distributed computing systems. Self healing is one property of autonomic computing, because it allows a system to find faults and correct them. Our model is built on this notion, as it goes through a similar cycle of observation, analysis, decision, action and verification. However, our model only uses controlled actions and does not allow autonomous changes without restriction.

Early studies on self-healing networks looked mainly at correlating events and discovering rules. Sterritt worked on self-healing in autonomic networks and developed a rule-discovery mechanism that relates alarm events [16]. With this method, system administrators could understand fault conditions and frame rules to determine faults. Because it made use of domain knowledge and human supervision, the process was easy to follow. The problem with a rule-based system is that it needs earlier experience of the same situation, and new failure patterns cannot be included.

Sharma and Sood [17] gave a full self-healing architecture of the autonomic kind for distributed computing. System logs were processed with data mining tools to produce healing rules, and a healing engine used these rules to select recovery actions. Their research showed that self-healing is not only about detecting faults but also about choosing the healing action and carrying it out. They connected monitoring information with the healing decision, which is a useful contribution. But the generated rules depend on the environmental conditions of the infrastructure or on failures with the same pattern, and this can reduce the flexibility of the architecture. In our framework, rules stay transparent, and Isolation Forest and XGBoost are added for anomaly detection and failure prediction.

With AIOps, artificial intelligence started being applied to the huge volume of operational data. An AIOps solution normally collects metrics, logs, traces, alerts and service data from infrastructure resources. This data is then analyzed to find abnormalities, predict incidents, correlate events and help in handling incidents. Alonso et al. studied optimization and prediction techniques for self-healing and self-learning applications in a trustworthy cloud continuum [7]. Their results showed that prediction together with self-management techniques gives many advantages in a distributed cloud environment. The method considered trustworthy operation and related predictions to the healing process, which was its main advantage. The cloud continuum, though, is a complex environment and needs a lot of data. We follow a similar approach in our framework for the infrastructure environment.

Abdullahi et al. proposed a Digital Twin for Predictive Maintenance in the Industrial Internet of Things [4]. In this approach, physical and computing resources are represented digitally for monitoring and maintenance. Visibility improves with the digital twin, since the current state of the system can be checked against the expected behavior, and so degradation can be detected before failure. On the other hand, the real environment and the digital twin have to be synchronized continuously, and extra effort for data management and computing may be needed. Our framework does not use a digital twin. It works with infrastructure telemetry and service data.

In self-healing, anomaly detection is an important step since a problem is usually preceded by the system moving away from its normal behavior. Conventional monitoring sets thresholds for CPU, memory, disk, response and availability. Setting a threshold is simple, but false positives are common when the normal workload changes. Sometimes a set of unusual values does not cross any threshold and gets missed. Unsupervised learning techniques help here, especially when there is little labeled failure data.

Liu, Ting and Zhou proposed Isolation Forest for detecting outliers efficiently [1]. Instead of building complex models for every class, it isolates observations by picking random features and partition values. An outlier is different from most of the other observations, so fewer partitions are needed to isolate it. This suits telemetry that is high-dimensional and keeps changing. In our solution we apply Isolation Forest to CPU, memory, disk, network, response-time, service-health and other measurements. The benefit is that abnormal behaviour can be detected even when not all potential failures are labelled. The drawback is that the anomaly score does not show the exact cause of the problem. For this reason we combine anomaly detection with failure prediction.

Logs are another important area in infrastructure research. They hold error messages, warnings, service events, restarts and similar information that metrics do not capture easily. Wang et al. introduced LightLog, a light temporal convolutional neural network to detect log anomalies in the edge environment [6]. The work detects anomalous log sequences while keeping in mind the limited resources at the edge, and it makes log analysis efficient by doing it near the data source. Log-based approaches still have problems. Log formats change, logs may be incomplete, and relating one isolated log to the overall state of the infrastructure is hard.

El-Shamy et al. studied anomaly and bottleneck detection in cloud data centers [18]. They pointed out that resource bottlenecks need to be detected in cloud environments. CPU, memory, storage and network conditions may interact, and then simple thresholding cannot be used. The research showed why monitoring and analysis of multiple metrics is necessary. However, detecting a bottleneck alone does not tell the reason for the problem or which recovery strategy to choose. Our framework moves in this direction.

Failure prediction tries to find deteriorating states before a clear disruption. Chen and Guestrin proposed XGBoost, a scalable gradient-boosting approach that uses decision trees [2]. It is

suitable for structured data, because it learns links between numerical and categorical features and handles non-linear interactions. It is also useful when infrastructure data has measurements of different scale and importance. We apply XGBoost to predict the probability of future failure and degradation using current and historical telemetry. It captures complex relationships in structured operational data well. Still, prediction depends on the quality and diversity of the input data, and accuracy may go down with rare events, changing workloads and incomplete labels. Hence in our framework prediction is used as a support for decision making.

Rouf et al. introduced InstantOps, in which prediction of system failures and identification of root causes are integrated for microservice cloud-native applications [5]. This is relevant to us since microservices bring dependencies among services, and it becomes hard to say whether the observed symptom is the cause of the problem. When prediction and diagnosis are done in one procedure, the delay between knowing about a problem and explaining it can be reduced. The paper focuses on failure analysis for cloud-native systems, which is its key advantage. Its weakness is that microservices may need traces, dependency graphs and a large amount of operational data.

The cause has to be found because an alert only tells that something is wrong and not which part needs to be fixed. Meng et al. researched service-oriented reliability modeling and autonomous reliability optimization for public cloud infrastructure [8]. They focused on how service reliability is related to automating decisions. These techniques can increase availability, but reliability optimization may involve complicated resource allocation and modeling of service dependencies. Our framework deals with detecting probable root causes for practical cases like CPU overload, memory exhaustion, disk exhaustion, service crashes and abnormal process behavior.

Some recent studies have looked at explainability and awareness of uncertainty in diagnosis. Duan et al. introduced LogEDL, which uses evidential deep learning to diagnose log anomalies [9]. Uncertainty has to be considered in operational decisions, more so when log patterns are not familiar, so the idea is interesting. In another work, Duan et al. look at few-shot diagnosis for AIOps to address the problem of having few labeled failure examples [19]. Both works show that real infrastructure settings may not have large training datasets. A drawback is that complex models can need more computation and tuning. For the diagnosis stage of our framework we use a Decision Tree along with metric and log based rules.

Automated recovery is what makes a self-healing system different from a monitoring system. A diagnosed state must be linked to a safe action. Restarting the affected service, restarting or recreating a failed container, cleaning up temporary resources, or making some other allowed adjustment to resources are examples of such actions. Meng et al. and Alonso et al. show why reliability analysis, prediction and optimal operation should be connected [7], [8]. Sanabria et al. address learning of recovery procedures for dynamic self-healing reactive systems [12]. In their study recovery is treated as a separate decision problem

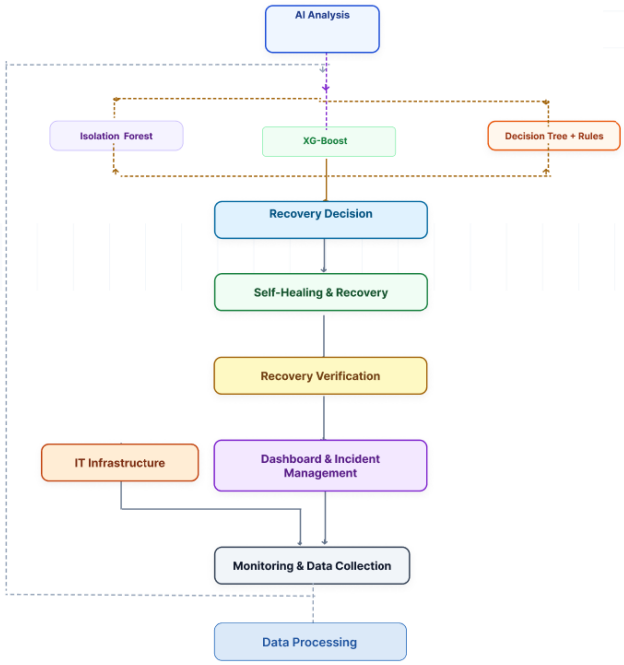


Fig. 1. Architecture of the Proposed AI-Based Self-Healing IT Infrastructure Framework

and not as something that automatically follows detection, as a wrong recovery procedure may cause more disruption.

Shetty et al. described the problems and guidelines in designing autonomous AI agents for the cloud [10]. Safety, control, observability and coordination of actions between cloud resources were among the problems they raised. Zota et al. [13] proposed a framework for creating agentic AIOps systems, and Sheth et al. studied agentic AI for adaptive response and resilience in cybersecurity [14]. These studies show that research is now moving towards systems that analyze incidents and also perform operations. But an agentic approach can lead to issues of unpredictable actions, permission, explainability and governance. Our framework does not assume that the agent is autonomous. AI models are used to detect, predict and diagnose incidents, and recovery is limited to certain actions only.

Jo et al. used deep reinforcement learning to study network reconfiguration for self-healing [11]. Their research shows that machine learning can choose a recovery strategy based on the changing conditions of the network. Reinforcement learning needs a proper environment, correct rewards, many iterations and safety measures, and we do not consider these points in our approach. Recent research on autonomic microservice management that combines agentic AI and the MAPE-K lifecycle [15] also confirms that feedback-based management is important and governance is needed.

From the works reviewed, self-healing research has moved from static rules and event correlation to monitoring, prediction, diagnosis and recovery with AI. A systematic review of self-healing with AI from the edge to the cloud was done in

2026 [20]. It noted challenges that are still not solved, such as validation, security, explainability, lack of data, scalability and human supervision. These are critical, because performing a recovery action and saying the system is healthy is not sufficient. The action has to be verified using some metrics.

Progress has been made in some areas, but most current approaches consider only one main task, like anomaly detection, log analysis, failure prediction, root cause analysis or recovery. The link between these phases often depends on different tools, datasets or infrastructures. A few approaches focus on autonomous operation but do not consider how unwanted actions can be prevented. So a general approach is required which integrates continuous monitoring, anomaly detection, failure prediction, probable root cause analysis, controlled recovery and recovery verification.

The AI-driven self-healing IT infrastructure framework described here deals with these issues by putting the following tools in a single process flow. Isolation Forest detects anomalies, XGBoost predicts future system failure, and a Decision Tree with rules explains the most likely cause of the problem. The recovery decision engine selects one of the pre-defined actions, and after recovery a verification step decides if the service health is restored. The Dashboard collects infrastructure statuses, alerts, incidents, predictions, recovery actions and healing history.

III. Methodology

We propose an AI based self-healing IT infrastructure that monitors infrastructure components all the time for anomalies, predicts failures in advance, finds out the cause of a failure and takes the corrective actions needed. The procedure is designed as stages, and after these the operators evaluate whether the issue has been solved. The system applies to web/API services, databases, workers and containers, but some corrective actions are not allowed for some classes.

A. Infrastructure Simulation

In the first phase the IT ecosystem is simulated, with the infrastructure elements that have to be monitored. These are applications such as web or API applications, databases, workers and containers, among others. The phase also creates the conditions in which normal operation and abnormal behaviour can be detected most easily. Changes in the IT environment are brought about by simulating variations in resource utilization, service health, response time and level of activity.

B. Monitoring and Data Collection

The monitoring component observes the simulated infrastructure and collects the relevant telemetry, which includes CPU usage, memory usage, disk usage, network bandwidth, latency of service response, container information and system logs. With this information the current state of the infrastructure is assessed, and it is also fed to the AI analysis that follows. Monitoring is meant to identify possible issues in the infrastructure before the service fails.

C. Data Processing and Feature Preparation

Telemetry data may come in different forms and scales. So in the data processing stage the telemetry is preprocessed before it is given as input to the AI models in the later stages. Metrics are collected here and transformed into features. Logs and signals are also used as complementary data sources for the root cause analysis done later. The stage is important because it keeps the input data consistent for the anomaly detection and failure prediction modules.

D. AI-Based Anomaly Detection

When preprocessing is complete, the framework detects unusual infrastructure behavior using the Isolation Forest algorithm. In our case it is used to find data samples that are clearly different from the general description of the data. Very few labels are available for unusual activity events, and outlier detection of this kind helps the model generalize better for the task. If unusual behavior is found, the corresponding telemetry is sent forward for processing in the next steps. A fairly simple algorithm fits the objectives of the project.

E. Failure Prediction

XGBoost is used in the failure prediction component to predict the probability of infrastructure failure in the future. Prediction does not end with finding that an anomaly has occurred. It also decides if the anomaly will cause an outage that makes the service inaccessible. The data gathered from infrastructure telemetry and processed earlier is given to the predictive model, which gives the probability of future failures. By predicting failures in the infrastructure, measures can be taken before the failure happens.

F. Root-Cause Analysis

After the abnormal state or a situation that may fail is identified, the model tries to find what could have caused it. For this, a decision tree is applied along with rule based systems for metrics and logs. Decisions are made using measurements from the infrastructure, health checks of the services and the related log signals, so that the detected state can be connected with its possible cause.

G. Controlled Self-Healing and Recovery

The recovery module picks the relevant predefined action based on the recognized condition and the probable reason. The framework is designed so that recovery is controlled and changes are not made to the production environment in an unrestricted or autonomous way. The actions depend on the problem recognized, and include recovering or restarting the affected service. This phase tries to minimize manual intervention while making sure automated operation is safe.

H. Recovery Verification and Incident Management

Executing an action is not the end of recovery. The framework checks the state of the service on its own to know if it is back to an acceptable level. Incidents detected, predictions, recovery actions, current state and healing events are recorded in the dashboard, so the operators can understand what the framework

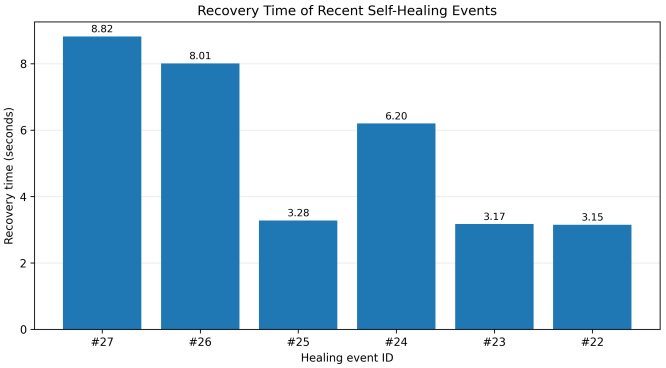


Fig. 2. Recovery Time of Recent Self-Healing Events

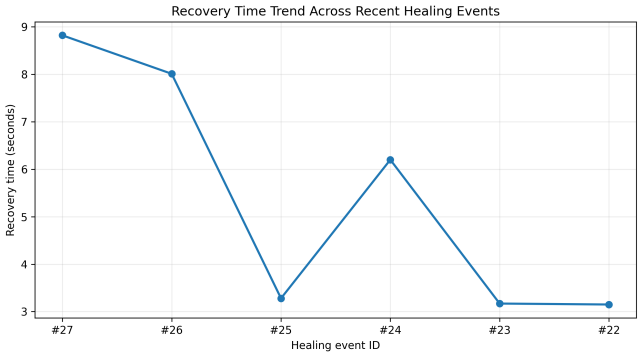


Fig. 3. Recovery Time Trend Across Recent Healing Events

has done and what it is doing currently.

IV. Result and Analysis

In this section we show how the proposed self-healing model was used and the results obtained from the dashboard. The visualizations give the recovery behavior, infrastructure condition and incident distribution recorded by the system. The web server, API server, database and worker services are monitored on the dashboard. In Fig. 2, the recovery time of the six healing events lies between 3.15 and 8.82 seconds. The average recovery time for these events is 5.44 seconds. From the distribution of events, recovery time changes depending on the service infrastructure and the incident that was handled. Fig. 3 below shows the same recovery events as a line graph. Time drops from 8.82 seconds to 3.28 seconds, goes up to 6.20 seconds, and then the last two events have shorter recovery times. Fig. 4 gives the distribution of the observed recovery times. Most of the events in Figure 4 are in the lower recovery time range, while two events have recovery times above eight seconds. This shows the variation in the healing process without assuming any particular recovery time. Fig. 5 gives the current measurements of the infrastructure. CPU usage is 25.63%, memory usage is 13.67%, container CPU usage is 0.23%, container memory usage is 57.55 MB, requests per second is 1.47, and API latency is 37.67 ms. These values show that the monitoring module is getting telemetry from the service. Fig. 6 shows the six incidents that were observed and recovered, according to the service affected. Three incidents were observed for the API server and three for the web server.

On the whole, the dashboard shows how monitoring, AI analysis, recovery and incident documentation are related. The values displayed are observations made on the current implementation, and they are not measurements of model accuracy. Larger experiments still have to be done to measure the performance of the system.

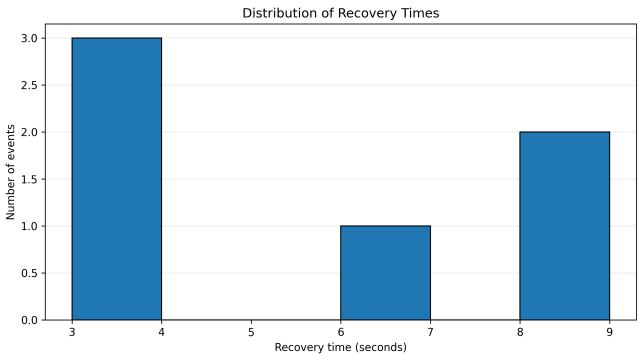


Fig. 4. Distribution of Recovery Times for Recorded Healing Events

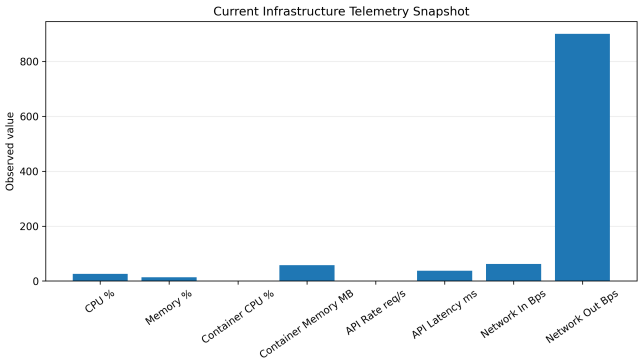


Fig. 5. Current Infrastructure Telemetry Snapshot

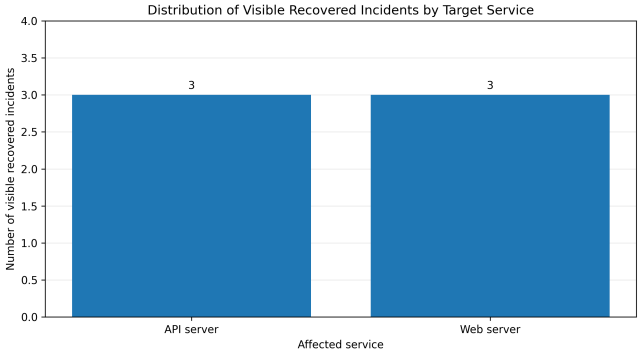


Fig. 6. Distribution of Visible Recovered Incidents by Target Service

V. Conclusion

The proposed AI-based self-healing IT infrastructure framework is meant to solve the problems in delivering services with guaranteed quality of service in distributed IT infrastructure. The framework does not depend on monitoring the infrastructure and responding to incidents manually. It uses self-contained

processes for monitoring, anomaly detection, failure prediction, root cause detection, recovery and verification of the actions completed. Isolation Forest is applied to detect anomalies in the infrastructure, and XGBoost is applied to predict failures that may occur, using the collected information. The Decision Tree algorithm, with rules based on metrics and logs, is applied to identify the potential root causes of incidents. Using this knowledge the framework runs the recovery process, and the result is checked to see whether the root causes were detected properly. The dashboard helps to visualize the current status, incidents, predictions and recovery processes. The framework can be expanded in different ways, for example monitoring more objects, using an adaptive recovery process, identifying root causes better or adding more log analysis.

REFERENCES

[1] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in Proc. 8th IEEE International Conference on Data Mining (ICDM), 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17.

[2] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[3] J. O. Kephart and D. M. Chess, "The Vision of Autonomic Computing," Computer, vol. 36, no. 1, pp. 41–50, 2003, doi: 10.1109/MC.2003.1160055.

[4] I. Abdullahi, S. Longo, and M. Samie, "Towards a Distributed Digital Twin Framework for Predictive Maintenance in Industrial Internet of Things (IIoT)," Sensors, vol. 24, no. 8, Art. no. 2663, 2024, doi: 10.3390/s24082663.

[5] R. Rouf, M. Rasolroveicy, M. Litoiu, S. Nagar, P. Mohapatra, P. Gupta, and I. Watts, "InstantOps: A Joint Approach to System Failure Prediction and Root Cause Identification in Microservices Cloud-Native Applications," in Proc. 15th ACM/SPEC International Conference on Performance Engineering (ICPE), 2024, doi: 10.1145/3629526.3645047.

[6] Z. Wang, J. Tian, H. Fang, L. Chen, and J. Qin, "LightLog: A Lightweight Temporal Convolutional Network for Log Anomaly Detection on the Edge," Computer Networks, vol. 203, Art. no. 108616, 2022, doi: 10.1016/j.comnet.2021.108616.

[7] J. Alonso, L. Orue-Echevarria, E. Osaba, J. López Lobo, I. Martinez, J. Diaz de Arcaya, and I. Etxaniz, "Optimization and Prediction Techniques for Self-Healing and Self-Learning Applications in a Trustworthy Cloud Continuum," Information, vol. 12, no. 8, Art. no. 308, 2021, doi: 10.3390/info12080308.

[8] S. Meng, L. Luo, X. Qiu, and Y. Dai, "Service-Oriented Reliability Modeling and Autonomous Optimization of Reliability for Public Cloud Computing Systems," IEEE Transactions on Reliability, vol. 71, no. 2, 2022, doi: 10.1109/TR.2022.3154651.

[9] Z. Duan et al., "LogEDL: Log Anomaly Detection via Evidential Deep Learning," Applied Sciences, vol. 14, no. 16, Art. no. 7055, 2024, doi: 10.3390/app14167055.

[10] M. Shetty et al., "Building AI Agents for Autonomous Clouds: Challenges and Design Principles," in Proc. 2024 ACM Symposium on Cloud Computing (SoCC), 2024, pp. 99–110, doi: 10.1145/3698038.3698525.

[11] S. Jo, J.-Y. Oh, Y. T. Yoon, and Y. G. Jin, "Self-Healing Radial Distribution Network Reconfiguration Based on Deep Reinforcement Learning," Results in Engineering, vol. 22, Art. no. 102026, 2024, doi: 10.1016/j.rineng.2024.102026.

[12] M. Sanabria, I. Dusparic, and N. Cardozo, "Learning Recovery Strategies for Dynamic Self-Healing in Reactive Systems," in Proc. 19th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), 2024, doi: 10.1145/3643915.3644097.

[13] R. D. Zota, C. Bărbulescu, and R. Constantinescu, "A Practical Approach to Defining a Framework for Developing an Agentic AIOps System," Electronics, vol. 14, no. 9, Art. no. 1775, 2025, doi: 10.3390/electronics14091775.

[14] A. Sheth et al., "AI Driven Self-Healing Cybersecurity Systems with Agentic AI for Adaptive Threat Response and Resilience," in Proc. 2025 IEEE Cloud Summit, 2025, doi: 10.1109/CloudSummit64795.2025.00030.

[15] M. Esposito, A. Bakhtin, N. Ahmad, M. Robredo, R. Su, V. Lenarduzzi, and D. Taibi, "Autonomic Microservice Management via Agentic AI and MAPE-K Integration," in Software Architecture: ECSA 2025 Tracks and Workshops, Lecture Notes in Computer Science, vol. 15982, Springer, 2026, pp. 105–118, doi: 10.1007/978-3-032-04403-$7_1$1.